Working Agents

A living research program on frameworks, principles and practices for working with AI agents

DAVE REMY · WORKING NOTE · DRAFT V0.1 · 6 NOVEMBER 2026

ABSTRACT

Working Agents is a series of working papers that develops a framework for working with AI agents: principles and strategies for operating a fleet of long-running agents, and a software development methodology for agent teams. Each paper mixes three things: theory, drawn from older disciplines and from current industry research; pragmatic advice a practitioner can apply; and evidence from a reference implementation, one person working with a fleet of agents organized into one or more teams. This note is the plan and index for the series. It explains how the papers are written, sorts them into two families (Agent Operations and Development Methodology), maps which papers build on which, and lists every published paper, draft and candidate as of 6 November 2026. The papers are living: each is checked every week against the reference implementation and the literature, and revised under one concept DOI, so a citation never breaks.

KEYWORDS AI agents, multi-agent systems, agent operations, software engineering process, living papers, research program

1 Why we write

The series develops a framework for working with AI agents: principles, strategies and practices for running a fleet of long-running agents, and a software development methodology for agent teams, Agent Scrum [1]. Each paper combines three sources. **Theory** comes from older disciplines, such as quality management and software process improvement, and from current industry research on agents. **Pragmatic advice** says what a practitioner can do with the idea, and what it costs. **Evidence** comes from a reference implementation and is measured, with its denominators and its limits.

The reference implementation is a fleet run by the author: nine named agents that run continuously and handle his life and work (a chief of staff, and agents for health, finance, communications, engineering, operations, customer support, research and marketing) [2], with the engineering agents building software in sprints. The number and roles are not the point. The framework should apply to a fleet with fewer or more agents, and each paper notes where a result depends on this particular setup. We write the papers for three reasons.

1.1 Writing for a reader exposes gaps in practice

A practice can work every day and still lack a step that nobody notices until they have to explain it to someone else. The clearest case so far is Kaizen for Agents [3], a paper on continuous improvement in agent teams. To describe the reference implementation's improvement loop, the paper had to show that the fixes proposed in sprint retrospectives worked. It sampled 20 retrospective issues that had been closed as done. Eighteen named a merged change. None recorded a measured effect. The team had a way to find problems and a way to fix them, but no step that checked whether a fix did what it was meant to do.

On the same day, 6 November 2026, the agents scheduled the missing step. Each retrospective fix will include a prediction: what changes, by how much, and how it is measured. A later sprint wrap records a verdict on each prediction that falls due: held, not held, or unmeasurable. Section 11.3 of the paper states its predictions before any data exists. For example, it predicts that at least half of the retrospective fixes filed in the next two weeks will include a prediction, against a baseline of none. The paper's public release waits about two weeks so that its next version can report whether the predictions held.

Writing the paper changed the process it describes. Teaching a practice is a way to learn it. The idea is old. Zinsser argued that writing is a way to learn a subject, not only a way to report on it [10].

1.2 Papers turn a working practice into a framework others can use

The reference implementation's working documents change every day, and they are written for the agents. A paper is written for an outside reader. It states a principle, explains why it holds, places it against prior work, shows what happened when we applied it, and gives it an identifier that others can cite. That turns one setup's habits into ideas another team can test in its own setup, and it gives the agents and the people who work with them one place to look for why the practice exists.

1.3 The evidence comes from one scenario

Each paper is a working paper or a working note based on one scenario: one person working with a fleet of AI agents, organized in the Scrum sense into one or more agent teams. Scaling these practices to larger human teams is a question for future work. None is a controlled study. Each reports what we did, what we measured and what we did not measure. Numbers come from the reference implementation's own records, and each paper says how they were counted. The theory and the industry research travel further than the evidence: read every measured finding as one case that may or may not hold in a different setup. Section 7 lists the limitations that apply to the whole series.

2 The two families

The papers fall into two families.

Agent Operations covers how to run a fleet of agents: long-lived sessions, scheduled jobs, memory, coordination between agents, and safety. It has no root paper. Persistent Agents on Claude Code [2] is one member, not the parent: it describes the whole fleet, but in less depth than a dedicated paper would. An

overview, provisionally titled "Running an Agent Fleet", may be written last, once enough members exist. It would then become the root.

Development Methodology covers how the fleet builds software. Agent Scrum [1] is its methodology paper and describes the whole method. Dedicated papers go deeper into one part of it, such as continuous improvement (Kaizen for Agents [3]) or choosing which tests to run in continuous integration (Agents Outrun Their Tests [4]). When a dedicated paper's approach has results, Agent Scrum gets a revision that summarizes the change in a paragraph and cites the dedicated paper for the detail. Agent Scrum does not wait for the dedicated paper to be published. It waits for results: its next version follows a pilot of a new way to estimate stories.

Each paper names the papers it builds on, in its repository and in its Zenodo record. A paper is never recorded as a version of another paper. Figure 1 shows the dependencies. Three cross the family line. Agent Scrum runs on the persistent agents that Persistent Agents describes. Kaizen for Agents discusses a nightly pass that corrects the agents' documents, which Dreams That Commit will describe in full. Agents Outrun Their Tests is about failures that test selection misses, a special case of the checks that report a pass without having tested anything, which the False passes candidate would cover.

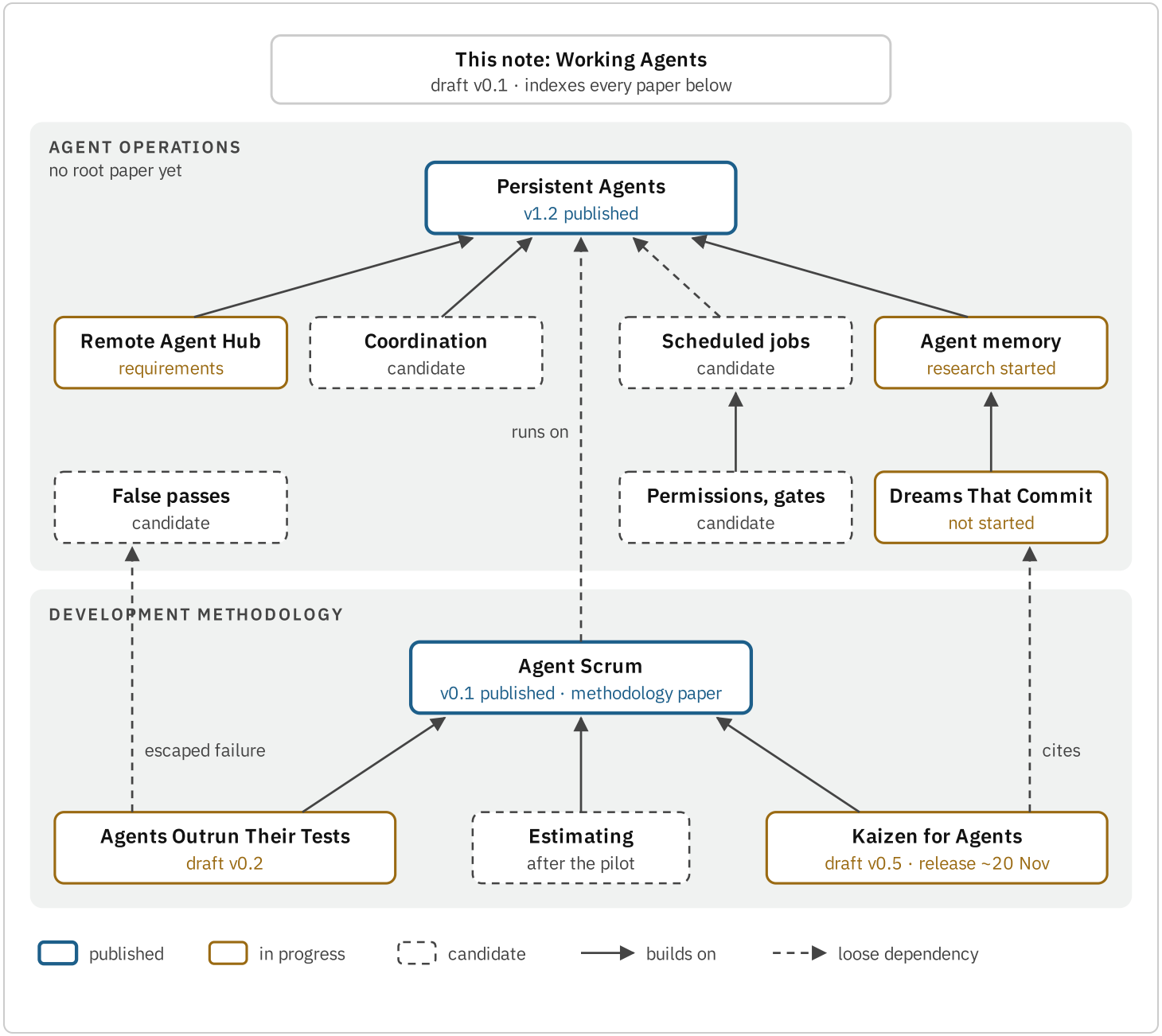


Figure 1. The series as of 6 November 2026. An arrow points from a paper to a paper it builds on. A dashed arrow is a looser link, such as a citation or a shared problem. The outline shows status: solid blue, published; amber, in progress; dashed, candidate. Not drawn: the three Development Methodology candidates that remain sections of Agent Scrum, and the possible Agent Operations overview (Table 1).

3 The papers

Table 1 lists every paper and candidate. A candidate is a practice with enough evidence that it may deserve a paper. It stays a line in this table until the author picks it to write. The Development Methodology candidates stay as sections of Agent Scrum until they have enough evidence to stand alone.

PAPER AND DOI	STATUS	BUILDS ON
AGENT OPERATIONS		

Persistent Agents on Claude Code [2]
`doi:10.5281/zenodo.23787644`
`v1.2: doi:10.5281/zenodo.24022879`
	Published, v1.2	

A Remote Agent Hub	**Requirements**	Persistent Agents
Agent memory (title to be decided)	**Research started**	Persistent Agents
Dreams That Commit: a nightly pass that corrects the agents' documents	**Not started**	Agent memory, Persistent Agents
Scheduled agent jobs: the job list, per-job tool allowlists, logged refusals	Candidate	Persistent Agents (loose)
False passes: checks that report a pass when nothing ran	Candidate	
Coordination: one agent routing messages, deduplication, broadcast rules	Candidate	Persistent Agents
Permissions and gates: allowlists, sends that need approval	Candidate	Scheduled agent jobs
Running an Agent Fleet (overview)	Possible, written last	All Operations members
DEVELOPMENT METHODOLOGY		

Agent Scrum (methodology paper) [1]
`doi:10.5281/zenodo.23771170`
`v0.1: doi:10.5281/zenodo.23771171`
	Published, v0.1; v0.2 after the estimation pilot	Persistent Agents (loose)

Kaizen for Agents [3]	**Draft v0.5**, release held to about 20 November for evaluation data	Agent Scrum, Persistent Agents; Dreams That Commit (loose)
Agents Outrun Their Tests: Targeted CI and the escaped failure [4]	**Draft v0.2**; the miss measurement is being collected	Agent Scrum; False passes (loose)
Estimating	Candidate, if the pilot's results outgrow a section	Agent Scrum
Sprint planning and refinement	Candidate, a section of Agent Scrum	Agent Scrum
Reviews: two reviewers from different model vendors	Candidate, a section of Agent Scrum	Agent Scrum
Impediments and escalation	Candidate, a section of Agent Scrum	Agent Scrum
BOTH FAMILIES		
Working Agents (this note) `DOI to be assigned`	**Draft v0.1**	Indexes every paper

Table 1. Every paper and candidate, as of 6 November 2026. For a published paper, the first DOI is the concept DOI, which always resolves to the latest version; the second is the DOI of the current version.

4 How papers stay current

Most published papers are fixed once they appear. A correction arrives as an erratum or as a new paper. This series updates its papers after publication, on purpose. This is uncommon for published work, but the way DOIs are assigned supports it (Section 4.2).

The reason is that the papers describe a system that keeps changing. A paper that was accurate in October can state a wrong sprint length, an old count or a retired tool by December, and a reader cannot tell which facts still hold unless someone checks. Living documents have precedents. Preprint servers such as arXiv keep numbered versions of a paper [7], and in medicine, living systematic reviews are updated as new evidence appears [6]. This series applies the same idea to working papers about a practice.

4.1 The weekly review

Each week the reference implementation's research agent runs a review in two parts.

Part A keeps published papers current. For each published paper, the review asks three questions. What changed in the system the paper describes? It reads the history of the files the paper's method depends on, checks for changes introduced by dedicated papers, and re-runs the commands the paper records. Is any stated fact now false, such as a version number, a sprint length, a count or a named tool? Has new prior work appeared that a reader would expect the paper to cite? The outcome is either a recorded "no change" or a new version.

Part B considers new papers. Which practices now have enough evidence to write up? The review reads the candidates in Table 1, adds candidates from the week's issues, incidents and merged changes, and brings the author at most two, each with its evidence, or reports that there is nothing new. Most weeks will not produce a paper.

4.2 Versions under one DOI

Each paper is published on Zenodo, an open research repository operated by CERN. On Zenodo, each version of a paper is a separate record with its own DOI and fixed files, linked to every other version [5]. A concept DOI stands for all versions and resolves to the latest one. For example, the Persistent Agents concept DOI, 10.5281/zenodo.23787644, resolves on 6 November 2026 to version 1.2, whose own DOI is 10.5281/zenodo.24022879. A reader who cites the concept DOI always reaches the current text. A reader who cites a version DOI gets exactly the text they read. An update never breaks a citation, which is why this model suits living papers.

4.3 Changes are visible, and publishing is the author's call

A new version adds a "Changes in vN.M" section near the top that says what changed and why, and the repository's README adds a line to its version list. A version that only corrects facts needs no new review. A version that changes a claim or adds a method gets a fresh review from a model by a different vendor than the one that drafted it.

Publishing any version is the author's decision, made version by version. Agents research, draft, review and prepare each paper; nothing becomes public until he approves it.

This note follows the same rules. It will have its own concept DOI, and it gets a new version when a paper ships, a candidate changes status, or the families change.

5 What ships with each paper

- **The paper**, as a web page and as a PDF printed from the same source.
- **A deck** of 14 to 16 slides for readers short on time, reusing the paper's figures, with the same numbers. It is published as a web page and a PDF.
- **An article** of 1,000 to 1,500 words for X and LinkedIn. It covers the problem, what we did, the measured result and what we would tell another team, and it links to the paper and its DOI. A new version that changes a headline number updates the article too.
- **A public repository** with the source, a README that lists each version, and a citation file.
- **Tools for implementers**, where they help: scripts, templates or agent skills that let a reader apply the paper's ideas in their own setup, kept in the paper's repository and versioned with it.
- **A license**: CC BY 4.0 [8]. Anyone may share and adapt the work, with attribution.
- **A Zenodo record** in the Working Agents community [9], which collects every paper and every version in one place.

The article became part of each release on 6 November 2026. Agent Scrum and Persistent Agents have their decks; their articles are still to be written.

6 How to follow and give feedback

- **Follow the community.** The Working Agents community page on Zenodo, zenodo.org/communities/working-agents, lists every paper and version as it is released.
- **Open an issue.** Each paper's repository takes feedback as GitHub issues: a wrong fact, a missing citation, a question about method, or a report of how a practice worked for another team. The published papers are at github.com/daveremy/agent-scrum and github.com/daveremy/persistent-agents. This note will have its own repository when it is published.
- **Read the articles.** The article for each paper appears on X (@daveremy) and LinkedIn.

Reports from other teams are the most useful feedback, because the largest limitation of the series is that it has one case. A change made in response to feedback is listed in the next version's "Changes" section.

7 Limitations

One case. Every paper describes one scenario: one person working with one fleet of agents, in one or more teams, on one set of codebases, in late 2026. How the practices change when several people share a fleet is future work. Results may depend on the person, the products, the models then available and the budget, and the series cannot separate those factors.

Self-reported. The author runs the system he describes, and AI agents both operate that system and help write about it. Numbers come from the reference implementation's own logs and repositories. Papers print

the commands behind their numbers where they can, and each draft is reviewed by a model from a second vendor. None of this replaces a study by someone outside the reference implementation.

Selection. The team chooses what to write, and it ranks candidates partly by how much evidence exists. Practices with good records get papers first. Practices that failed quietly, or left little record, may never get one. The two families are also our own classification and may change.

Living text. Revisions keep the papers accurate, but a claim quoted from one version may be gone from the next. Cite the version DOI when the exact wording matters.

R References

1. Remy, D. *Agent Scrum: A sprint methodology for continuous software development by AI agent teams, with observations from 324 sprints.* Working paper, version 0.1, 2026. doi:10.5281/zenodo.23771171. daveremy.github.io/agent-scrum
2. Remy, D. *Persistent Agents on Claude Code.* Working note, version 1.2, 2026. doi:10.5281/zenodo.24022879 (all versions: doi:10.5281/zenodo.23787644). daveremy.github.io/persistent-agents
3. Remy, D. *Kaizen for Agents: Continuous improvement in an AI software team, read against Toyota, software process improvement and compound engineering.* Working paper, draft v0.5, 2026. Forthcoming.
4. Remy, D. *Agents Outrun Their Tests: Targeted CI and the escaped failure.* Working paper, draft v0.2, 2026. Forthcoming.
5. Zenodo. Manage versions. help.zenodo.org/docs/deposit/manage-versions
6. Elliott, J. H., Turner, T., Clavisi, O. et al. Living systematic reviews: an emerging opportunity to narrow the evidence-practice gap. *PLoS Medicine* 11(2): e1001603, 2014. doi:10.1371/journal.pmed.1001603
7. arXiv. Submit a new version of a work. info.arxiv.org/help/replace.html
8. Creative Commons. Attribution 4.0 International (CC BY 4.0). creativecommons.org/licenses/by/4.0
9. Working Agents community. Zenodo. zenodo.org/communities/working-agents
10. Zinsser, W. *Writing to Learn.* Harper & Row, 1988.

Created in collaboration with AI.